Reporte de Aprendizaje Práctico-Experimental Nro. 3

1. Datos Generales

Integrantes	Mateo Rojas, Leonardo Sánchez, Tayron Morales.
Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Sincronización de Procesos: Exclusión Mutua, Semáforos y Problemas Clásicos de Concurrencia
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de de 2026 09 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- Implementar una barrera de sincronización con variables de condición que detenga a los hilos hasta que todos alcancen el punto de encuentro.
- Implementar mecanismos de exclusión mutua utilizando mutex y cerrojos (locks) para proteger secciones críticas en programas multihilo, garantizando que no se produzcan condiciones de carrera al acceder a recursos compartidos.

- Aplicar semáforos como herramienta de sincronización para controlar el acceso a recursos limitados y coordinar la ejecución de hilos según restricciones específicas de orden y disponibilidad.
- Resolver el problema clásico del Productor-Consumidor utilizando buffers compartidos y mecanismos de sincronización que aseguren que el productor no escriba en un buffer lleno y el consumidor no lea de un buffer vacío.
- Implementar la solución al problema de Lectores-Escritores, distinguiendo entre las variantes con prioridad para lectores prioridad para escritores, analizando las implicaciones de cada enfoque en términos de inanición y concurrencia.

3. Materiales y reactivos:

Guia practica

Lista de ejercicios

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Ejercicio 1. Control de acceso a un estacionamiento — Semáforo contador

Objetivo: Aplicar semáforos contadores para controlar el acceso concurrente a un recurso limitado.

Situación:

Un estacionamiento dispone de **4 espacios**. Llegan 10 vehículos de manera concurrente. Cada vehículo debe esperar si no existe un espacio disponible. Cuando un vehículo sale, libera el espacio para otro vehículo.

Procedimiento

1. Identifique el recurso compartido.
2. Determine la cantidad máxima de vehículos que pueden permanecer dentro del estacionamiento.
3. Cree un semáforo contador inicializado en 4.
4. Cree un proceso vehiculo(id).
5. El vehículo deberá solicitar un espacio antes de ingresar.
6. Si no existen espacios disponibles, deberá permanecer bloqueado.
7. Una vez que obtenga un espacio, deberá simular su permanencia.
8. Al salir, deberá liberar el espacio mediante signal().

9. Cree 10 procesos vehículo.
10. Analice qué sucede cuando llegan más de 4 vehículos simultáneamente.

Pseudocódigo

```
SEMÁFORO espacios = 4


PROCEDIMIENTO vehiculo(id)
   escribir("Vehículo ", id, " llega al estacionamiento")

   wait(espacios)

   escribir("Vehículo ", id, " ingresa al estacionamiento") estacionar(id)

   esperar_tiempo()

   escribir("Vehículo ", id, " sale del estacionamiento")

   signal(espacios)

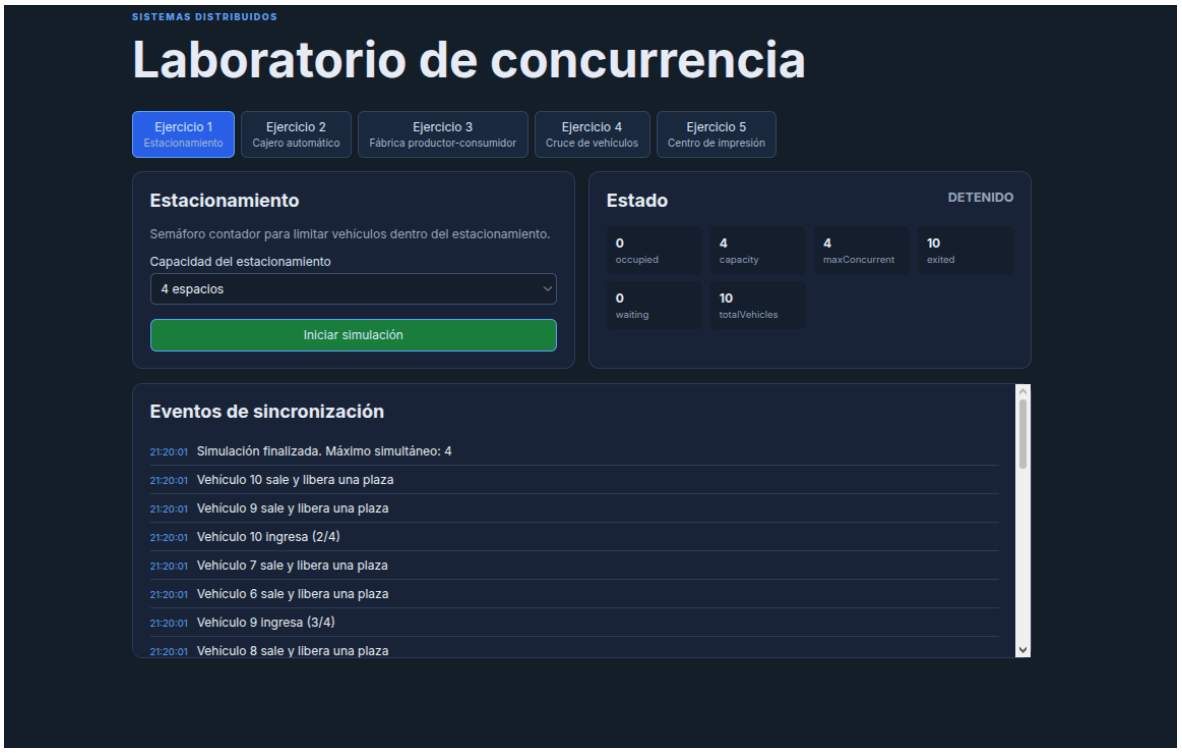
FIN PROCEDIMIENTO


INICIO

   PARA i = 1 HASTA 10 HACER
      crear_proceso(vehiculo, i)
   FIN PARA

FIN
```

Vista del control de acceso a un estacionamiento:





Pregunta de análisis:

¿Qué ocurriría si el semáforo se inicializara en 1? ¿Y si se inicializara en 10?

Si el semáforo se inicializa en 1 un vehículo podría ingresar al estacionamiento una vez, los otros vehículos tendrían que esperar obligadamente hasta que el vehículo se esté estacionado salga y libere el permiso del semáforo.

Ejercicio 2. Cajero automático compartido — Exclusión mutua

Objetivo: Implementar exclusión mutua para evitar condiciones de carrera sobre un recurso compartido.

Situación:
Una cuenta bancaria tiene un saldo inicial de $1000. Tres procesos realizan operaciones simultáneamente sobre la misma cuenta:

- Proceso 1: retiro de $300.
- Proceso 2: retiro de $250.
- Proceso 3: depósito de $200.

El saldo debe modificarse de forma segura.

Procedimiento

1. Identifique la variable compartida.
2. Determine qué operaciones pueden ejecutarse simultáneamente.
3. Cree un mutex para proteger el saldo.
4. Antes de consultar o modificar el saldo, el proceso debe adquirir el mutex
5. Realice la operación correspondiente.
6. Libere el mutex después de actualizar el saldo.
7. Cree los tres procesos.
8. Ejecute el programa varias veces.
9. Analice qué podría ocurrir si se elimina el mutex.
10. Compare el comportamiento con y sin exclusión mutua.

Pseudocódigo

```
saldo = 1000

MUTEX mutex_saldo = 1


PROCEDIMIENTO retirar(id, cantidad)

   lock(mutex_saldo)

   escribir("Proceso ", id, " intenta retirar $", cantidad)

   SI saldo >= cantidad ENTONCES

      saldo = saldo - cantidad

      escribir("Retiro realizado") escribir("Saldo
      actual: $", saldo)
```

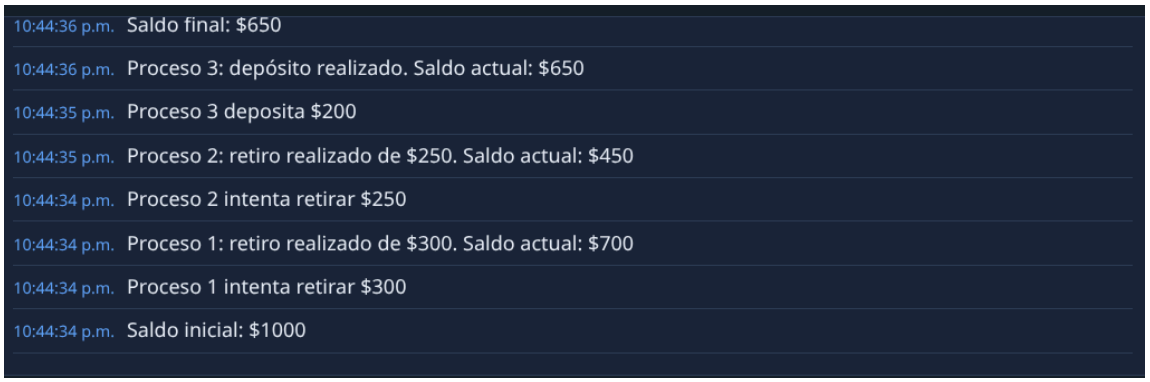
```
    SINO
        escribir("Saldo insuficiente")
    FIN SI
    unlock(mutex_saldo)
FIN PROCEDIMIENTO

PROCEDIMIENTO depositar(id, cantidad)
    lock(mutex_saldo)
    escribir("Proceso ", id, " deposita $", cantidad)
    saldo = saldo + cantidad
    escribir("Saldo actual: $", saldo)
    unlock(mutex_saldo)
FIN PROCEDIMIENTO

INICIO
    crear_proceso(retirar, 1, 300)
    crear_proceso(retirar, 2, 250)
    crear_proceso(depositar, 3, 200)
FIN
```

Vista del cajero automático:





Pregunta de análisis:

¿Por qué el acceso a saldo debe protegerse con un mutex?

El acceso al saldo debe protegerse con un mutex porque varios procesos pueden intentar leer y modificar la misma variable al mismo tiempo. El mutex permite que solo un proceso acceda al saldo a la vez, evitando condiciones de carrera y asegurando que el valor final sea correcto y consistente.

Ejercicio 3. Fábrica de ensamblaje — Productor-Consumidor

Objetivo: Implementar el patrón productor-consumidor utilizando una cola limitada y semáforos.

Situación:

Una fábrica tiene una línea de producción con capacidad para almacenar **5 piezas** temporalmente.

- Los productores fabrican piezas.
- Los consumidores toman piezas para ensamblarlas.
- Un productor no puede colocar una pieza si el almacén está lleno.

- Un consumidor no puede retirar una pieza si el almacén está vacío.

Procedimiento

1. Cree una cola con capacidad máxima de 5 elementos.
2. Cree un semáforo espacios inicializado en 5.
3. Cree un semáforo elementos inicializado en 0.
4. Cree un mutex para proteger la cola.
5. Implemente el proceso productor.
6. El productor deberá esperar un espacio disponible.
7. Debe adquirir el mutex antes de modificar la cola.
8. Debe insertar una pieza.
9. Debe liberar el mutex.
10. Debe indicar que existe una nueva pieza.
11. Implemente el proceso consumidor.
12. El consumidor deberá esperar hasta que exista una pieza.
13. Debe adquirir el mutex antes de retirar una pieza.
14. Debe retirar la pieza.
15. Debe liberar el mutex.
16. Debe indicar que existe un espacio disponible.

Pseudocódigo

```
COLA  almacen
CAPACIDAD = 5

SEMÁFORO espacios = 5
SEMÁFORO elementos = 0

MUTEX mutex_almacen = 1


PROCEDIMIENTO productor(id)

    REPETIR 5 VECES

     pieza = fabricar_pieza()

     wait(espacios)

     lock(mutex_almacen)

     insertar(almacen, pieza)
```

```
        escribir("Productor ", id,
                " agregó pieza ", pieza)

        unlock(mutex_almacen)

        signal(elementos)

    FIN REPETIR

  FIN PROCEDIMIENTO


PROCEDIMIENTO consumidor(id)

    REPETIR 5 VECES

        wait(elementos)
        lock(mutex_almacen) pieza = extraer(almacen)
        escribir("Consumidor ", id, "
                retiró pieza ", pieza)

        unlock(mutex_almacen)

        signal(espacios)

        ensamblar(pieza)

    FIN REPETIR

FIN PROCEDIMIENTO


INICIO

    PARA i = 1 HASTA 2 HACER
        crear_proceso(productor, i)
    FIN PARA
```

```
PARA i = 1 HASTA 2 HACER
    crear_proceso(consumidor, i)
FIN PARA

FIN
```

Vista de la fábrica de ensamblaje:



Pregunta de análisis:

¿Qué sucede si espacios se inicializan en 0? ¿Qué sucede si elementos se inicializan en 5?

Si los espacios se inicializan en 0, los productores se bloquean porque no hay permisos para colocar piezas en la cola. Como consecuencia, no se producen elementos y los consumidores también quedan esperando; la simulación se bloquea.

Si los elementos se inicializan en 5, los consumidores creen que hay cinco piezas disponibles aunque la cola está vacía. Intentan retirar piezas y quedan bloqueadas hasta que los productores agreguen elementos. La sincronización queda incorrecta, porque el semáforo indica piezas que realmente no existen.

Ejercicio 4. Cruce de vehículos — Control de acceso en una dirección

Objetivo: Coordinar procesos que comparten un recurso y evitar que dos grupos incompatibles accedan simultáneamente.

Situación:

Un puente permite circular en ambos sentidos, pero solamente puede existir **una dirección de circulación activa a la vez**.

Pueden existir varios vehículos cruzando hacia el norte, pero no puede haber vehículos hacia el norte y hacia el sur simultáneamente.

Procedimiento

1. Identifique el recurso compartido.
2. Defina una variable que indique la dirección actual.
3. Mantenga un contador de vehículos que están cruzando.
4. Proteja las variables mediante un mutex.
5. Un vehículo deberá verificar la dirección antes de entrar.
6. Si existen vehículos en dirección contraria, deberá esperar.
7. Si puede entrar, incrementará el contador correspondiente.
8. Después de cruzar, disminuirá el contador.
9. Cuando no queden vehículos, el puente podrá cambiar de dirección.
10. Analice el problema de inanición.

Pseudocódigo

```
MUTEX mutex = 1
direccion = NINGUNA
vehiculos_norte = 0
vehiculos_sur = 0


PROCEDIMIENTO vehiculo_norte(id) lock(mutex)

   MIENTRAS direccion == SUR HACER esperar()
   FIN MIENTRAS

   direccion = NORTE

   vehiculos_norte = vehiculos_norte + 1 unlock(mutex)


   escribir("Vehículo ", id,
         " cruza hacia el NORTE")

   cruzar_puente(id)
```

```
lock(mutex)

vehiculos_norte = vehiculos_norte - 1
SI vehiculos_norte == 0 ENTONCES direccion = NINGUNA
FIN SI

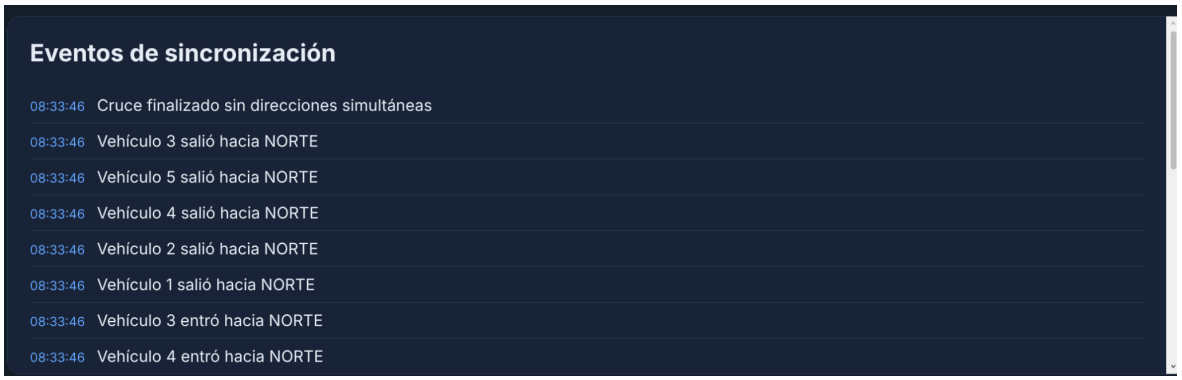
unlock(mutex)

FIN PROCEDIMIENTO
```

El estudiante deberá completar el procedimiento: PROCEDIMIENTO vehiculo_sur(id)
siguiendo la misma lógica.

Vista del Cruce de vehículos:





Reto: modificar el algoritmo para evitar que una dirección monopolice el puente.

Ejercicio 5. Centro de impresión — Sincronización de procesos

Objetivo: Utilizar semáforos para establecer un orden determinado de ejecución entre procesos.

Situación:

Un centro de impresión tiene tres procesos:

- Diseño
- Revisión
- Impresión

Para cada documento debe cumplirse obligatoriamente:

DISEÑO → REVISIÓN → IMPRESIÓN

No se puede revisar un documento que todavía no ha sido diseñado y tampoco se puede imprimir antes de que haya sido revisado.

Procedimiento

1. Identifique los procesos participantes.
2. Cree un semáforo para cada etapa.
3. Permita inicialmente que el proceso de diseño pueda ejecutarse.
4. Diseño deberá habilitar a revisión.
5. Revisión deberá habilitar a impresión.
6. Impresión deberá habilitar nuevamente a diseño.
7. Repita el ciclo para 3 documentos.
8. Verifique que el orden siempre sea correcto.
9. Analice qué ocurre si se elimina uno de los signal().

Pseudocódigo

```
SEMÁFORO turno_diseno = 1
SEMÁFORO turno_revision = 0
SEMÁFORO turno_impresion = 0



PROCEDIMIENTO diseno()
REPETIR 3 VECES

    wait(turno_diseno)

    escribir("Diseño prepara documento")

    crear_documento() signal(turno_revision)

FIN REPETIR
```

```
    FIN PROCEDIMIENTO


PROCEDIMIENTO revision()

      REPETIR 3 VECES

        wait(turno_revision)

        escribir("Revisión verifica documento")

        revisar_documento() signal(turno_impresion)

    FIN REPETIR

FIN PROCEDIMIENTO


PROCEDIMIENTO impresion()

      REPETIR 3 VECES

        wait(turno_impresion)

        escribir("Impresión imprime documento")

        imprimir_documento() signal(turno_diseno)

        FIN REPETIR


FIN PROCEDIMIENTO


INICIO

    crear_proceso(diseno)
```

crear_proceso(revision)

crear_proceso(impresion)

FIN

Vista del centro de impresión:





Link del GitHub: https://github.com/Sebaslsst/Sistemas_Distribuidos_APE3.git

6. Resultados esperados:

- Comprobación de las distintas formas de cómo los hilos se comunican.
- Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes
- El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la práctica

Ejercicio	Resultado esperado	¿Cumple según el código?	Justificación
1. Estacionamiento	Máximo 4 vehículos dentro.	Sí, con capacidad 4.	El semáforo limita el ingreso y libera un espacio cuando sale un vehículo. La capacidad predeterminada es 4.
2. Cajero	Saldo final de $650.	Sí.	El mutex protege las operaciones sobre el saldo: $1000 − $300 − $250 + $200 = $650.
3. Productor-Consumidor	Todos los productos son consumidos y el almacén no supera 5 piezas.	Sí.	La cola tiene capacidad 5. Los semáforos controlan los espacios y las piezas disponibles, y se distribuye el consumo de todas las piezas producidas.
4. Puente	Una sola dirección activa a la vez.	Sí.	Los vehículos esperan cuando el puente está ocupado en sentido contrario. La dirección puede cambiar cuando queda vacío.
5. Centro de impresión	Diseño → Revisión → Impresión.	Sí.	Los semáforos habilitan cada etapa después de finalizar la anterior y repiten el ciclo para 3 documentos.

7. Preguntas de Control:

- **¿Qué es una condición de carrera?**

  Es cuando dos o más hilos compiten por recursos o procesos y acceden a recursos compartidos y al final depende quien termine de ejecutarse primero [2][3].

- **Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas**

  La diferencia radica en el concepto de la propiedad que establece que el mutex coloca el hilo que lo ha bloqueado tiene los permisos para desbloquearlo, en cambio un semáforo binario no tiene un propietario como tal debido a esto no se pueden usarse indistintamente por la liberación por terceros si se intenta hacer esto con un mutex el sistema operativo genera un error o un comportamiento indefinido, también la inversión de prioridad que los mutex que se suelen incorporar con diferentes mecanismos para el sistema operativo lo cual un hilo del alta prioridad se quede bloqueado indefinidamente[1].

- **Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?**
  Un interbloqueo es un estado que es permanente que en un conjunto de procesos se queda bloqueado por cada uno de los retienen un recursos para poder adquirir otros, se debe cumplir cuatro condiciones como la exclusión mutua, su retención y al espera, sin expropiación y una espera circular [3].

8. Conclusiones

En conclusión, la implementación de barreras de sincronización, mutex y cerrojos permitió comprender la importancia de coordinar la ejecución de los hilos y proteger las secciones críticas, evitando condiciones de carrera y accesos simultáneos inadecuados a los recursos compartidos.

Asimismo, la aplicación de semáforos y la resolución del problema del Productor-Consumidor permitieron controlar el acceso a recursos limitados, garantizando que las operaciones de lectura y escritura se realicen de forma coordinada según la disponibilidad del búfer.

Finalmente, el análisis del problema de Lectores-Escritores permitió identificar las diferencias entre la prioridad para lectores y escritores, comprendiendo sus efectos sobre la concurrencia y el riesgo de inanición, aspectos fundamentales para desarrollar programas multihilo eficientes y seguros.

9. Declaración de IA

Para el desarrollo de esta actividad se utilizó ChatGPT como herramienta de apoyo académico y guía de información durante la realización de la práctica. La herramienta fue empleada para facilitar la comprensión de los conceptos a la sincronización de hilos mediante barreras y variables de condición, la implementación de mecanismos de exclusión mutua mediante mutex y cerrojos, así como el uso de semáforos para controlar el acceso a recursos compartidos y prevenir condiciones de carrera. Además, permitió orientar la comprensión de los procedimientos de implementación y funcionamiento de estos mecanismos, contribuyendo al análisis y cumplimiento de los objetivos planteados en la práctica.

Prompt utilizado:
"Explícame de forma clara y sencilla la sincronización de hilos, incluyendo la implementación de barreras de sincronización mediante variables de condición, el uso de mutex y cerrojos (locks) para proteger secciones críticas y evitar condiciones de carrera, y la aplicación de semáforos para controlar el acceso a recursos compartidos. Describe cómo funcionan estos mecanismos, sus diferencias, sus ventajas y cómo se pueden implementar en programas multihilo para cumplir los objetivos de una práctica académica".

10. Bibliografía

[1] H. Kurdi y V. Thayananthan**, "Authentication mechanisms for IoT system based on distributed MQTT brokers: review and challenges," *Procedia Computer Science*, vol. 194, pp. 132–139, 2021, doi: 10.1016/j.procs.2021.10.066.

[2] A. Lercher, "Managing API Evolution in Microservice Architecture," en Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24), Lisboa, Portugal, abr. 2024, pp. 3–5, doi: 10.1145/3639478.3639800.

[3] S. S. Gill et al., "Modern computing: Vision and challenges," Telematics and Informatics Reports, vol. 13, art. 100116, mar. 2024, doi: 10.1016/j.teler.2024.100116.

[4] Semana 3. Guaman, Jose. 2026